МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ

З ЛАБОРАТОРНОЇ РОБОТИ №6

з дисципліни «Функціональне програмування»

на тему: «Замикання та декоратори як HOF»

Варіант 6: «Повторні спроби файлової операції»

Виконав:
студент групи ФЕП-23
Корж Олексій

Перевірив:
проф. Свелеба С. А.

Львів – 2026

1. Мета роботи

Опанувати механізм створення та функціонування замикань (closures) у Python, збереження стану через ключове слово nonlocal; навчитися проєктувати параметризовані декоратори як фабрики декораторів; реалізувати декоратори retries (повторні спроби з backoff), call_stats (збір статистики викликів), timed (вимірювання часу) та cached_with_ttl (кешування з TTL та інвалідацією); забезпечити інжекцію залежностей та збереження метаданих функцій через functools.wraps.

2. Теоретичні відомості

Замикання (Closure): функція, що утримує посилання на змінні зі своєї лексичної області видимості (free variables), навіть після завершення виконання зовнішньої функції. Ключове слово nonlocal дозволяє модифікувати змінні зовнішнього контексту, реалізуючи інкапсульований стан.

Декоратор: функція вищого порядку, що приймає функцію та повертає її обгорнуту версію, розширюючи її поведінку без зміни вихідного коду.

functools.wraps: копіює метадані вихідної функції (__name__, __doc__, __wrapped__) на функцію-обгортку, запобігаючи втраті інтроспекції та коректності документації.

Інжекція залежностей (Dependency Injection): передача зовнішніх функцій (годинника, sleep, логера) через аргументи декоратора у вигляді Callable. Це робить юніт-тести абсолютно детермінованими, ізольованими від системного таймера та надзвичайно швидкими.

3. Постановка завдання (Варіант 6)

Тема варіанта 6: Повторні спроби файлової операції.

Обов’язкові компоненти: Замикання-логер (make_logger), лічильник викликів (make_call_counter), симулятор нестабільного файлового читання (make_file_simulator), декоратор повторів (@retries з backoff), декоратор статистики (@call_stats), @timed, @cached_with_ttl.

Спеціальні вимоги: Імітація тимчасових помилок читання/запису дисків (OSError) без реального диску; підміна функцій sleep_fn та logger; перевірка збереження метаданих.

4. Хід роботи та лістинг програми

Модуль замикань (closures.py)

```
from __future__ import annotations

from collections.abc import Callable


def make_logger() -> tuple[
    Callable[[str], None],
    Callable[[], list[str]],
    Callable[[], None],
]:
    logs: list[str] = []

    def log(msg: str) -> None:
        logs.append(msg)

    def get_logs() -> list[str]:
        return list(logs)

    def clear() -> None:
        logs.clear()

    return log, get_logs, clear


def make_call_counter() -> tuple[
    Callable[[], int],
    Callable[[], int],
    Callable[[], None],
]:
    count = 0

    def increment() -> int:
        nonlocal count
        count += 1
        return count

    def get_count() -> int:
        return count

    def reset() -> None:
        nonlocal count
        count = 0

    return increment, get_count, reset


def make_file_simulator(
    fail_attempts: int,
    content: str,
    error_msg: str = "Temporary disk I/O error",
) -> tuple[Callable[[str], str], Callable[[], int]]:
    attempts = 0
```

```
    def read_file(filepath: str) -> str:
        nonlocal attempts
        attempts += 1
        if attempts <= fail_attempts:
            raise OSError(f"{error_msg} for {filepath} (attempt {attempts})")
        return content

    def get_attempts() -> int:
        return attempts

    return read_file, get_attempts
```

Модуль параметризованих декораторів (decorators.py)

```
from __future__ import annotations

from collections.abc import Callable
from functools import wraps
from typing import Any, ParamSpec, Protocol, TypeVar, cast

P = ParamSpec("P")
R = TypeVar("R")
R_co = TypeVar("R_co", covariant=True)


class WrappedFn(Protocol[P, R_co]):
    __name__: str
    __doc__: str | None
    __wrapped__: Callable[..., Any]

    def __call__(self, *args: P.args, **kwargs: P.kwargs) -> R_co: ...


class StatsRecord:
    def __init__(self) -> None:
        self.total_calls: int = 0
        self.successes: int = 0
        self.failures: int = 0
        self.total_duration: float = 0.0

    @property
    def avg_duration(self) -> float:
        if self.total_calls == 0:
            return 0.0
        return self.total_duration / self.total_calls


class StatsFn(Protocol[P, R_co]):
    stats: StatsRecord
    __name__: str
    __doc__: str | None
    __wrapped__: Callable[..., Any]

    def __call__(self, *args: P.args, **kwargs: P.kwargs) -> R_co: ...


class CachedFn(Protocol[P, R_co]):
    __name__: str
```

```
    __doc__: str | None
    __wrapped__: Callable[..., Any]

    def invalidate(self) -> None: ...

    def __call__(self, *args: P.args, **kwargs: P.kwargs) -> R_co: ...


def retries(
    max_attempts: int = 3,
    delay: float = 0.1,
    backoff: float = 2.0,
    exceptions: tuple[type[BaseException], ...] = (Exception,),
    sleep_fn: Callable[[float], None] | None = None,
    logger: Callable[[str], None] | None = None,
) -> Callable[[Callable[P, R]], WrappedFn[P, R]]:
    if max_attempts < 1:
        raise ValueError("max_attempts must be >= 1")

    def decorator(fn: Callable[P, R]) -> WrappedFn[P, R]:
        @wraps(fn)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            current_delay = delay
            for attempt in range(1, max_attempts + 1):
                try:
                    return fn(*args, **kwargs)
                except exceptions as err:
                    if logger:
                        logger(f"Attempt {attempt}/{max_attempts} failed: {err}")
                    if attempt == max_attempts:
                        raise
                    if sleep_fn:
                        sleep_fn(current_delay)
                    current_delay *= backoff
            raise RuntimeError("Unreachable")

        return cast(WrappedFn[P, R], wrapper)

    return decorator


def call_stats(
    clock_fn: Callable[[], float] | None = None,
) -> Callable[[Callable[P, R]], StatsFn[P, R]]:
    get_time = clock_fn if clock_fn is not None else (lambda: 0.0)

    def decorator(fn: Callable[P, R]) -> StatsFn[P, R]:
        stats_obj = StatsRecord()

        @wraps(fn)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            stats_obj.total_calls += 1
            t_start = get_time()
            try:
                result = fn(*args, **kwargs)
                stats_obj.successes += 1
                return result
            except BaseException:
                stats_obj.failures += 1
                raise
            finally:
```

```
                t_end = get_time()
                stats_obj.total_duration += max(0.0, t_end - t_start)

        cast(Any, wrapper).stats = stats_obj
        return cast(StatsFn[P, R], wrapper)

    return decorator


def timed(
    sink: Callable[[str, float], None] | None = None,
    clock_fn: Callable[[], float] | None = None,
) -> Callable[[Callable[P, R]], WrappedFn[P, R]]:
    get_time = clock_fn if clock_fn is not None else (lambda: 0.0)

    def decorator(fn: Callable[P, R]) -> WrappedFn[P, R]:
        @wraps(fn)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            t0 = get_time()
            try:
                return fn(*args, **kwargs)
            finally:
                elapsed = max(0.0, get_time() - t0)
                if sink:
                    sink(fn.__name__, elapsed)

        return cast(WrappedFn[P, R], wrapper)

    return decorator


def cached_with_ttl(
    ttl: float = 60.0,
    clock_fn: Callable[[], float] | None = None,
) -> Callable[[Callable[P, R]], CachedFn[P, R]]:
    get_time = clock_fn if clock_fn is not None else (lambda: 0.0)

    def decorator(fn: Callable[P, R]) -> CachedFn[P, R]:
        cache: dict[tuple[Any, ...], tuple[R, float]] = {}

        @wraps(fn)
        def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
            key = (args, tuple(sorted(kwargs.items())))
            now = get_time()
            if key in cache:
                val, ts = cache[key]
                if now - ts <= ttl:
                    return val

            val = fn(*args, **kwargs)
            cache[key] = (val, now)
            return val

        def invalidate() -> None:
            cache.clear()

        cast(Any, wrapper).invalidate = invalidate
        return cast(CachedFn[P, R], wrapper)

    return decorator
```

Демонстраційний застосунок (app.py)

```
from __future__ import annotations

from closures import make_file_simulator, make_logger
from decorators import call_stats, retries, timed


def main() -> None:
    logger, get_logs, _ = make_logger()

    simulated_reader, get_attempts = make_file_simulator(
        fail_attempts=2,
        content='{"status": "success", "payload": "data_chunk_42"}',
    )

    logged_times: list[tuple[str, float]] = []

    def sink(fn_name: str, elapsed: float) -> None:
        logged_times.append((fn_name, elapsed))

    clock = 100.0

    def fake_clock() -> float:
        nonlocal clock
        clock += 0.05
        return clock

    sleeps: list[float] = []

    def fake_sleep(d: float) -> None:
        sleeps.append(d)

    @call_stats(clock_fn=fake_clock)
    @timed(sink=sink, clock_fn=fake_clock)
    @retries(
        max_attempts=4,
        delay=0.1,
        backoff=2.0,
        exceptions=(OSError,),
        sleep_fn=fake_sleep,
        logger=logger,
    )
    def fetch_file(path: str) -> str:
        return simulated_reader(path)

    print("Запуск операції читання файлу...")
    result = fetch_file("/var/data/input.json")
    print("Результат читання:", result)
    print("Всього спроб файлового симулятора:", get_attempts())
    print("Зафіксовані затримки (sleep):", sleeps)
    print("Логи замикання-логера:")
    for log_msg in get_logs():
        print("  -", log_msg)

    stats = fetch_file.stats
    print(
        f"\nСтатистика викликів: {stats.total_calls} всього, "
        f"{stats.successes} успіхів, {stats.failures} помилок"
    )
```

```
if __name__ == "__main__":
    main()
```

Тести замикань (tests/test_closures.py)

```
import pytest
from closures import make_call_counter, make_file_simulator, make_logger


def test_logger_closure() -> None:
    log, get_logs, clear = make_logger()
    assert get_logs() == []
    log("msg 1")
    log("msg 2")
    assert get_logs() == ["msg 1", "msg 2"]
    clear()
    assert get_logs() == []


def test_call_counter_closure() -> None:
    inc, get_cnt, reset = make_call_counter()
    assert get_cnt() == 0
    assert inc() == 1
    assert inc() == 2
    assert get_cnt() == 2
    reset()
    assert get_cnt() == 0


def test_file_simulator_closure() -> None:
    read_fn, get_attempts = make_file_simulator(2, "payload_ok")
    assert get_attempts() == 0

    with pytest.raises(OSError):
        read_fn("file.txt")
    assert get_attempts() == 1

    with pytest.raises(OSError):
        read_fn("file.txt")
    assert get_attempts() == 2

    assert read_fn("file.txt") == "payload_ok"
    assert get_attempts() == 3
```

Тести повторних спроб (@retries) (tests/test_retries.py)

```
from collections.abc import Callable

import pytest
from closures import make_file_simulator, make_logger
from decorators import retries


def test_retries_immediate_success() -> None:
    calls = 0
```

```
    @retries(max_attempts=3)
    def ok_fn(x: int) -> int:
        nonlocal calls
        calls += 1
        return x * 2

    assert ok_fn(5) == 10
    assert calls == 1


def test_retries_with_simulated_file(
    sleep_recorder: tuple[Callable[[float], None], Callable[[], list[float]]],
) -> None:
    fake_sleep, get_sleeps = sleep_recorder
    log_fn, get_logs, _ = make_logger()

    sim_read, get_attempts = make_file_simulator(fail_attempts=2, content="data")

    @retries(
        max_attempts=3,
        delay=0.1,
        backoff=2.0,
        exceptions=(OSError,),
        sleep_fn=fake_sleep,
        logger=log_fn,
    )
    def fetch() -> str:
        return sim_read("a.txt")

    assert fetch() == "data"
    assert get_attempts() == 3
    assert get_sleeps() == [0.1, 0.2]
    assert len(get_logs()) == 2


def test_retries_exhausted() -> None:
    sim_read, get_attempts = make_file_simulator(fail_attempts=5, content="data")

    @retries(max_attempts=2, exceptions=(OSError,))
    def fetch() -> str:
        return sim_read("a.txt")

    with pytest.raises(OSError):
        fetch()
    assert get_attempts() == 2


def test_preserves_metadata() -> None:
    def original_fn(x: int) -> int:
        return x + 1

    wrapped = retries(max_attempts=3)(original_fn)
    assert wrapped.__name__ == "original_fn"
    assert wrapped.__wrapped__ is original_fn
```

Тести статистики викликів (@call_stats) (tests/test_stats.py)

```
from collections.abc import Callable

import pytest
from decorators import StatsRecord, call_stats


def test_call_stats_tracking(
    fake_clock: tuple[Callable[[], float], Callable[[float], None]],
) -> None:
    clock, advance = fake_clock

    @call_stats(clock_fn=clock)
    def compute(x: int, should_fail: bool = False) -> int:
        advance(0.5)
        if should_fail:
            raise ValueError("fail")
        return x * 10

    assert compute(2) == 20
    assert compute(3) == 30

    with pytest.raises(ValueError):
        compute(4, should_fail=True)

    stats: StatsRecord = compute.stats
    assert stats.total_calls == 3
    assert stats.successes == 2
    assert stats.failures == 1
    assert stats.total_duration == 1.5
    assert stats.avg_duration == 0.5
```

Тести таймінгу (@timed) (tests/test_timed.py)

```
from collections.abc import Callable

from decorators import timed


def test_timed_decorator(
    fake_clock: tuple[Callable[[], float], Callable[[float], None]],
) -> None:
    clock, advance = fake_clock
    records: list[tuple[str, float]] = []

    def sink(name: str, duration: float) -> None:
        records.append((name, duration))

    @timed(sink=sink, clock_fn=clock)
    def task() -> str:
        advance(0.25)
        return "done"

    res = task()
    assert res == "done"
    assert len(records) == 1
    assert records[0] == ("task", 0.25)
```

Тести кешування (@cached_with_ttl) (tests/test_cached.py)

```
from collections.abc import Callable

from decorators import cached_with_ttl


def test_cached_ttl_and_invalidation(
    fake_clock: tuple[Callable[[], float], Callable[[float], None]],
) -> None:
    clock, advance = fake_clock
    calls = 0

    @cached_with_ttl(ttl=10.0, clock_fn=clock)
    def expensive(x: int) -> int:
        nonlocal calls
        calls += 1
        return x * x

    assert expensive(4) == 16
    assert calls == 1

    assert expensive(4) == 16
    assert calls == 1

    advance(5.0)
    assert expensive(4) == 16
    assert calls == 1

    advance(6.0)  # total 11 > 10.0 TTL expired
    assert expensive(4) == 16
    assert calls == 2

    # Invalidation
    invalidate_fn = expensive.invalidate
    invalidate_fn()
    assert expensive(4) == 16
    assert calls == 3
```

5. Результати виконання та тестування

Запуск демонстраційної програми app.py

```
$ python app.py
Запуск операції читання файлу...
Результат читання: {"status": "success", "payload": "data_chunk_42"}
Всього спроб файлового симулятора: 3
Зафіксовані затримки (sleep): [0.1, 0.2]
Логи замикання-логера:
  - Attempt 1/4 failed: Temporary disk I/O error for /var/data/input.json (attempt 1)
  - Attempt 2/4 failed: Temporary disk I/O error for /var/data/input.json (attempt 2)

Статистика викликів: 1 всього, 1 успіхів, 0 помилок
```

Результати виконання тестів pytest

```
$ pytest -q
..........                                                               [100%]
10 passed in 0.55s
```

Статична перевірка типів mypy --strict

```
$ mypy --strict .
Success: no issues found in 9 source files
```

Перевірка форматування ruff та black

```
$ ruff check .
All checks passed!

$ black --check .
All done! ✨ 🍰 ✨
9 files would be left unchanged.
```

6. Висновки

У лабораторній роботі № 6 реалізовано повноцінний набір замикань та параметризованих декораторів. Використання інжекції годинника, sleep_fn та логера дозволило протестувати складні сценарії відновлення після збоїв (retries) без затримок у тестах. Усі декоратори зберігають сигнатури та метадані завдяки functools.wraps та Protocol, забезпечуючи 100% сумісність із mypy --strict.